Department of Chemical Engineering

00560413 – AI Fluency and Agentic Methods for Engineers

Fall Semester, 2026-7

Lectures: Mon. 08:30 – 10:30 Ulman 803
Exercises: Mon. 10:30 – 12:30 Ulman 501 & 703 (in parallel)

Lecturer: Assoc. Prof. Alon Grinberg Dana
Reception hours: by advanced coordination via email

TA: Mr. Ori Goldfryd
Reception hours: by advanced coordination via email

TA: Mr. Ramzy Ayan
Reception hours: by advanced coordination via email

Prerequisites:

02340128 – Introduction to Computing (Python)
00540330 – Simulation Laboratory, for students of the Department of Chemical Engineering
For other students, an equivalent simulation course approved in advance

For all students, basic software engineering proficiency is required.
Verifying that prerequisites are met is the student's responsibility.

Academic Points: 2.5

Course Description and Goals:

This course prepares students to practice their discipline alongside AI agents, treating an agent as a component of engineering workflows. Students establish the scientific basis (neural networks, how LLMs differ from deterministic numerical methods), operate a modern professional scientific software stack (version control, code review, type checking, testing, CI), and perform key operations by hand before delegating, so they can verify and debug what the agent does. Each student builds a structured input layer driving a process simulation and an agent that operates on it: proposing design changes, invoking the simulator as a deterministic tool, reading results, and iterating toward an improved design under explicit constraints and human-in-the-loop checkpoints, targeting yield, energy efficiency, and/or selectivity. Every graduate ships at least one working agentic system that solves a real engineering or research problem in their own field. The course is offered by the Department of Chemical Engineering and open across the Technion, while case studies are adapted to other domains as needed. Prerequisites for BSc students: 02340128 (Python); 00540330 (Simulation Lab) for Chemical

Engineering, or an equivalent pre-approved simulation course for other departments. Prerequisites for all: A basic software engineering proficiency is required. The course uses open-source and free-tier tooling wherever possible.

Learning outcomes:

At the end of the course the students will be able to:

1. Explain the central principle of sound AI-assisted engineering and its consequences for engineering judgment and accountability.
2. Explain what a large language model is, how it differs from deterministic numerical methods, and why its probabilistic nature imposes specific verification requirements.
3. Operate a modern professional scientific software stack (version control, code review, type checking, testing, including CI) at a level appropriate for collaborative engineering work.
4. Engineer prompts and context, including ubiquitous-language glossaries and structured specifications, to align human and agent on shared engineering concepts.
5. Formulate an engineering design or optimization problem for an agent: defining the design space, objective, and constraints, and coupling the agent to a deterministic simulator for evaluation.
6. Design and build a custom agentic system for a real engineering or research problem, including a structured text-based input layer coupled to a simulation, tool use, retrieval, and human-in-the-loop checkpoints.
7. Verify and validate AI-assisted work by writing evaluation suites and identifying hallucinations.
8. Reason about cost and model selection as engineering trade-offs, and write appropriate AI-usage disclosure reflecting what was delegated and where human judgment intervened

Topics:

	Topic	Subjects
1	Intro	Syllabus; the 10x engineer; what an agent is; the environment (WSL, harnesses, anaconda, git, Linux terminal); the three postures; the for-loop test
2	LLM science	NN; LLM; transformer; next-token prediction; context window; scaling; emergence
3	Agent (harness + LLM)	Harness: the loop, tools, memory; skills and hooks; model selection and cost; AI fluency and the 4 Ds; automation bias; LLM council; adversarial development; subagents and context isolation
4	Containment	Prompt injection; untrusted tool output; the two walls; policy versus enforcement; fail-open defaults; the canary probe and why it needs a control; agent environment files
5	Context engineering	What goes in the window and why; memory versus the context window; agent environment files as a versioned artifact; domain glossaries; retrieval cheapest-first (grep, stuffing, BM25, routers, embeddings); file formats; writing for agents
6	Elicitation: being grilled into a specification	Specs vs. prompts; being interviewed by the agent; goals, non-goals and anti-goals; qualifying a ground against the six criteria; data governance and what may be shared with a model

7	Tests as the control surface	Test-driven development; unit, functional and evaluation tests; specification-derived development; deep modules; CI; provenance of computed results
8	Connecting tools	Protocol servers versus wrapping a solver; APIs; structured output and schemas; simulator-driven input and output; hallucinated packages
9	Human in the loop	Interactions with agents; drift and inflection; intervention points as a declared artifact; observability and structured logging; diagnosing from the log alone; inflection-point ledgers and input yield; when not to use an agent; token economics
10	Discernment and verification in depth	Verification and validation; evaluation suites; calibration as a measurable property (Brier score, reliability diagrams, coverage–accuracy); physical sanity checks; red-teaming your own work
11	Open	System 1 decision models; Reserved
12	Open	Reserved
13	Showcase and defense	Project demonstrations; live defense; course retrospective; **attendance mandatory**

Grade Composition:

Component	**Weight**
Setup (W2)	10%
Working product	40%
In-class demo and live defense (W13)	25% (**attendance mandatory**)
Written report	20%
Peer-review quality	5%
Total	*100%*

Project Setup (W2).
It has four parts: the sandbox check script passing on your own machine; a merged pull request with green CI; the course environment activating and a run completing; and five short written answers on why each check exists: for example, why does setting a sandbox flag to true not prove that anything is enforced, and why does a canary probe need a control?

The project milestones.

	Due	**Deliverable**
P1	W2	Problem statement; GitHub user registered with course staff; first merged PR
P2	W4	First agent-authored commit
P3	W6	Agent environment file; domain glossary; one written skill; written specification; (last chance to consider project revision)
P4	W8	One real engineering tool connected and in use; Specification-derived tests green in CI
P5	W10	Declared intervention points; structured logging; one induced failure diagnosed from the log alone; first evaluation run
P6	W12	Evaluation suite; red-team exchange closed with a peer; canonical run; demo; tagged release; defense pack

The live defense.

In W13 (**attendance mandatory**) students demonstrate their system and then defend it. Attendance in W13 is mandatory for all students: those who defend are selected at random, so everyone must be present and ready.

Peer review.
Each student reviews the work of assigned peers and files findings as issues on their repositories. The grade is for the quality of the review.

Reproducibility check.
Each submission is verified from a clean clone: build, run tests, execute, and check.

Resources:

Before W1, students should refresh themselves in Python. **Students need their own laptop.** The Technion provides a per-student API key with a small cap for Google LLMs. A free-model lane is available as well.

Use of AI tools in this course:

Using AI tools is required in this course, not merely permitted. The subject matter is how to direct them competently. What is assessed is therefore not whether you used an agent, but whether you directed it, verified it, and can account for it. Students must **be able to defend their project.** You will be asked about your own work in W13, in person. Submitting work you cannot account for is integrity failure.

Values and academic integrity:

In this course we work together to learn how to direct AI agents on real engineering problems, in order to acquire tools that can serve us to solve real world problems. For this end, we hold a number of core values:

- **Integrity and professionalism –** Students are expected to act with integrity and professionalism, to direct and verify their own work rather than forwarding unreviewed agent output, and to be active contributors if work is done in a group. Passing off unverified output as verified will be dealt with in a proper disciplinary manner.
- **Professional education and growth –** We all learn and grow, both personally and professionally. We take responsibility for our professional education and encourage others in the course in the learning process while maintaining integrity. Learning groups are seen in a positive light.
- **Safety and health –** We maintain our physical and mental health. If we do not feel well or have any illness symptoms, we will avoid arriving to class or reception hours physically.

Accommodations for students with special needs:

For any personal need, please contact the lecturer or a TA as early in the semester as possible.

Additional useful information:

The course website site contains lecture and exercise materials. Course code and templates live in the course GitHub organization; students work in their own forks.

Good luck!